Arrays in C

- Arrays are used to store a group of data of the same type in memory
- Contiguous memory locations are used to store arrays in memory
- When we want to refer a number of similar variables with the same name, arrays are useful
- Instead of storing 30 different names of students, one name *student* can be used with a different index or subscript
- Thus the elements of an array are accessed using a subscript also called an index

One dimensional Arrays

- Arrays can be one dimensional (vectors), two-dimensional (matrices) or multi-dimensional
- A list of items can be given one variable name with only one subscript and that variable is called one dimensional array
- The general format for declaring one-dimensional arrays is as below

  data-type array-name [size]

  where data-type can be any C data-type like int, float or char

  array-name is any valid variable name

  and size is the maximum number of array variables

  The subscript values are between 0 and size-1

  If we try to access a variable outside this range, an error will occur

Initalizing one dimensional arrays

- There are two ways to initialize arrays

1. Initializing in the declaration itself
2. Using for statement

Examples

int num [5] = { 1,4,6,8,9};

It means num[0] = 1, num[1] = 4,…,num[4] = 9

float marks [3] = { 67.5,70.0,86.4]

char vowels[5] = {‘a’, ‘e’, ‘i’, ‘o’, ’u’}

float values [5] = { 34.5, 15.6, 78.4}

Only three values are initialized; the remaining two values are set to zero by the compiler

Intializing with for statement

- for(i = 0; i<= 4,i++)

  A[i] =0;

- for (k=0; k<=50; k++)

  scanf (“%f”, &marks[k]);

Thus it is convenient to use for statement to read a large number of values stored with an array name

Algorithm 1: Maximum element

- Algorithm for finding the maximum element in a list:

procedure max (a_1, a_2, …, a_n: integers)
max := a_1
for i := 2 **to** n
  if max < a_i **then** *max* := a_i

{*max* is the largest element}

Maximum element running time

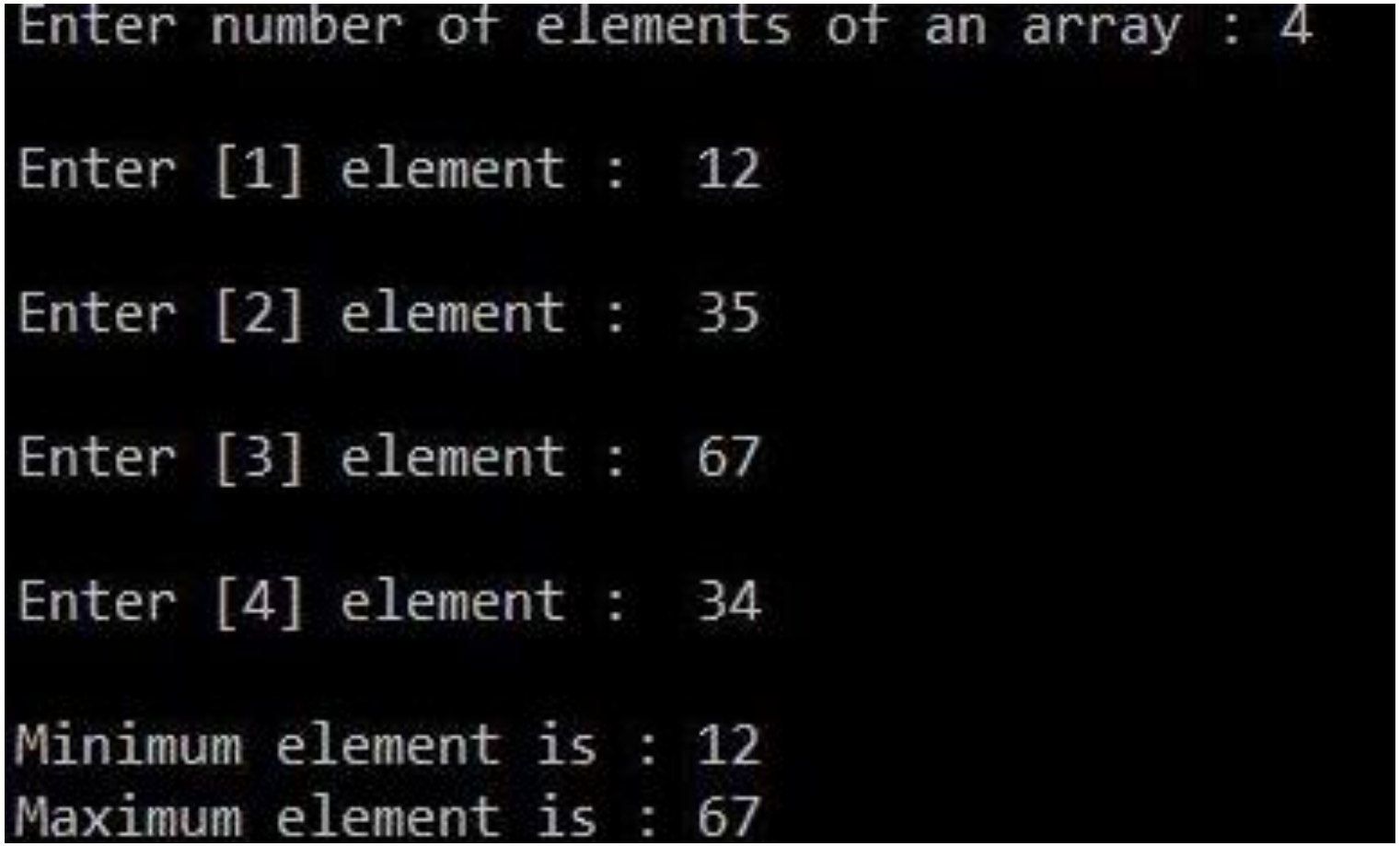
- Computing time is proportional to the number of comparisons; each comparison can be a step

- If the list has *n* elements, in the worst case it takes *n* “steps”
- So the computing time is O(n) (order of n)

Finding Maximum and minimum in an array

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
int n,ar[100],i,max,min;
system("cls");   //system() is a library function of stdlib. h header file
// This function is used to run system/ command prompt commands and here cls is a //command to clear the output screen
printf("\n Enter number of elements of an array : ");
scanf("%d",&n);
for(i=0; i<n; i++)
{ printf("\n Enter [%d] element : ",i+1);
scanf("%d",&ar[i]); }
i=0; min=ar[i]; max=ar[i];
for(i=1; i<n; i++) // min and max
{ if(ar[i]<min)
min=ar[i];
if(ar[i]>max)
max=ar[i]; }
printf("\n Minimum element is : %d",min);
printf("\n Maximum element is : %d",max);
return 0;
}
```

Output: Maximum and Minimum



Algorithm 2: Linear search

- Given a list, find a specific element in the list

procedure linear_search (x: integer; a_1, a_2, …, a_n: integers)
i := 1
while ($i \leq n$ and $x \neq a_i$)
  i := i + 1
if $i \leq n$ **then** *location* := i
else *location* := 0

{*location* is the subscript of the term that equals x, or it is 0 if x is not found}

Linear search running time

- What is the computing time?

- If the list has *n* elements, in the worst case it takes *n* “steps”
- Thus the computing time for linear search is O(n)

Algorithm 3: Binary search

- Given a list, find a specific element in the list
  - List MUST be sorted!
- Each time it iterates through, it cuts the list in half

procedure binary_search (x: integer; a_1, a_2, …, a_n: increasing integers)
i := 1 { i is left endpoint of search interval }
j := n { j is right endpoint of search interval }
while $i < j$
begin
  m := $\lfloor (i+j)/2 \rfloor$ { m is the point in the middle }
  if x > a_m **then** i := m+1
  else j := m
end
if $x = a_m$ **then** *location* := m
else *location* := 0

{*location* is the subscript of the term that equals x, or it is 0 if x is not found}

Sorting algorithms

- Given a list, put it into ascending or descending order
  - Numerical, lexicographic (dictionary order), etc.

  There are a number of sorting techniques
  - Bubble Sort
  - Insertion Sort
  - Selection Sort
  - Quick Sort
  - Merge Sort

Algorithm : Bubble sort

- One of the most simple sorting algorithms
  - Also one of the least efficient
- It takes successive elements and “bubbles” them up the list

procedure bubble_sort (a_1, a_2, …, a_n)
for i := 1 **to** n-1
  for j := 1 **to** n-i
    if $a_j > a_j$+1
      then interchange a_j and a_j+1
{ a_1, …, a_n are in increasing order }

Algorithm : Bubble sort Example

- Consider 6 element list: 5 4 15 3 7 2

for i=1 to 5

for j=1 to 6-i

if a[j] > a[j+1]

then interchange a[j] and a[j+1]

I =1

j =1 to 5 *5 4* 15 3 7 2

j=1 a[1] & a[2] are compared *4 5* 15 3 7 2

j=2 a[2] & a[3] are compared 4 *5 15* 3 7 2

j=3 a[3] & a[4] are compared 4 5 *3 15* 7 2

j=4 a[4] & a[5] are compared 4 5 3 *7 15* 2

j=5 a[5] & a[6] are compared 4 5 3 7 *2 15*

Final Output:2 3 4 5 7 15

Bubble sort running time

- Bubble sort algorithm:

  for i := 1 **to** n-1

  for j := 1 **to** n-i

  if $a_j > a_j$+1

  then interchange a_j and a_j+1

- Outer for loop does n-1 iterations
- Inner for loop does
  - n-1 iterations the first time
  - n-2 iterations the second time
  - …
  - 1 iteration the last time
- Total: (n-1) + (n-2) + (n-3) + … + 2 + 1 = (n^2-n)/2
  - We can say that's "about" n^2 time

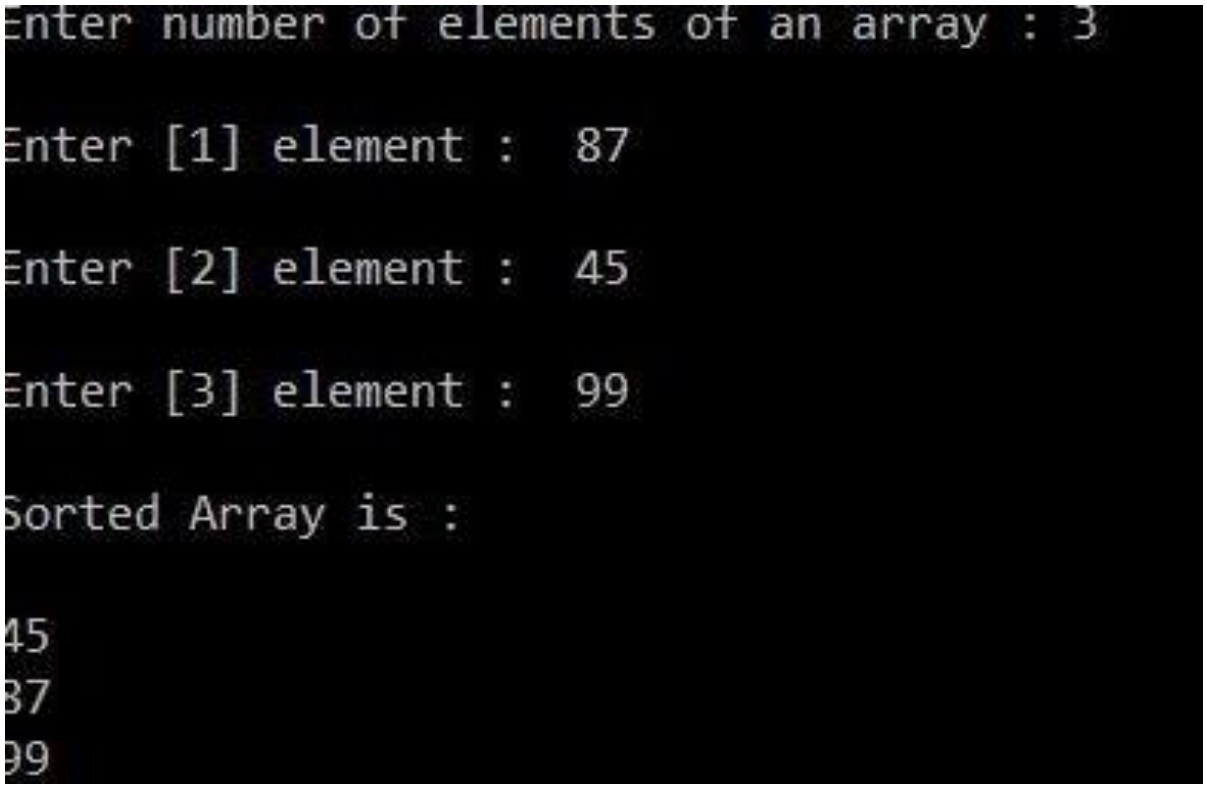
Bubble sort

```
/* Sort array using bubble sort */
#include<stdio.h>
int main()
{
int n,ar[100],i,j,temp;
system("cls");
printf("\n Enter number of elements of an array : ");
scanf("%d",&n);
for(i=0; i<n; i++)
{
printf("\n Enter [%d] element : ",i+1);
scanf("%d",&ar[i]);
}
```

Bubble sort contd..

```
for(i=0; i<n-1; i++)
{
for(j=0; j<n-1-i; j++)
{
if(ar[j]>ar[j+1])
{
temp=ar[j];
ar[j]=ar[j+1];
ar[j+1]=temp;
}
}
}
printf("\n Sorted Array is : \n");
for(i=0; i<n; i++)
{
printf("\n %d",ar[i]);
}
return 0;   }
```

Output: Bubble sort



Two Dimensional Arrays: Matrices

- A Table of values can be represented by a two dimensional (2-D) array; A2-D array is also called a matrix
- Example:A Table of Marks of students Marks[4][6]

Roll No.	Subj1	Subj2	Subj3	Subj4	Subj5
1	87	77	75	65	90
2	85	67	95	75	73
3	73	79	85	67	81
4	68	67	84	76	66

There are four rows and 6 columns

Rows represent marks of each student in 5 different subjects

Columns represent marks of 4 different students in one subject

Initalizing two dimensional arrays

- There are two ways to initialize 2-D arrays

1. Initializing in the declaration itself
2. Using for statements

Examples

```
int table [2] [3] = { 1,4,6,8,9,10}; or equivalently
int table [2][3] = { {1,4,6}, {8,9,10}} ;or
int table [2][3] = { {1,4,6},
                     {8,9,10}};
```

It means table[0][0] = 1, table[0][1] = 4
table[0][2]= 6, table[1][0]= 8, table[1][1]= 9
table[1][2]= 10

```
int table[2][3] =  { {1,1},
                     {2}};
```

will initialize the first two elements of first row to 1, the first element of second row to 2 and all other elements to 0

Declaration of Matrices

- The general format for declaring two-dimensional arrays is as below

  data-type array-name [rowsize] [columnsize]

  where data-type can be any C data-type like int, float or char

  array-name is any valid variable name

  and rowsize is the maximum number of rows and columnsize is the maximum number of columns

  The subscript values are between 0 and size-1

  If we try to access a variable outside this range, an error will occur

Intializing, Reading and Writing with for statements

- ```
  for(i = 0; i< 4,i++)
    for (j=0; j<5; j++)
      A[i][j] =0;
  ```
  Matrix A[4][5] is initialized to 0

- ```
  for (r=0; r<3; r++)
   for (c=0;c<3; c++)
  scanf ("%f", &marks[r][c]);
  ```
  3x3 matrix marks values are input

- ```
  for (r=0; r<3; r++)
    for (c=0;c<3; c++)
  printf("%f", marks[r][c]);
  ```
  3x3 matrix marks values are input

  3x3 matrix marks values are printed
  ```

Matrix Operations

- For addition and subtraction, the dimension of both the matrices must be equal
- Addition of two matrices is obtained by adding corresponding elements of each of the two matrices
- Subtraction of two matrices is obtained by subtracting from each element of first matrix, the corresponding element of the second matrix
- For matrix multiplication of two matrices, the number of columns of first matrix and the number of rows of second matrix must be equal
- The elements of product matrix are obtained by adding the product of each row element of first matrix with each column element of second matrix

Example of Matrix Operations

Let A = [1 2 3 and B [1 1 1

4 5 6] 2 2 2]

A+ B = [2 3 4

6 7 8]

A - B = [0 1 2

2 3 4]

A = [1 1 1 B = [2 2 2 First element of A* B = 1*2+1*3+1*4

1 1 1 3 1 2 Second element of A* B = 1*2+1*1+1*1

1 1 1] 4 1 2] Third element of A* B = 1*2+1*2+1*2

Matrix operations

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int a[10][10],b[10][10],c1[10][10],i,j,r,c,ch,k;
        clrscr();
printf("\n\nHOW MANY ROWS AND COLUMN S YOU WANT TO ENTER IN  MATRICES ? n");
        scanf("%d%d",&r,&c);
        printf("\nROWS ARE : %d \tCOLUMNS ARE : %d",r,c);
        printf("\nENTER FIRST MATRIX\t");
        for(i=0;i<r;i++)
        {
                for(j=0;j<c;j++)
                        scanf("%d",&ai][j]);
        }

```

Print Input Matrices

```
printf("\nENTER SECOND MATRIX\t");
        for(i=0;i<r;i++)
        {
                for(j=0;j<c;j++)
                        scanf("%d",&b[i][j]);
        }
        printf("\nFIRST MATRIX IS :\n");
        for(i=0;i<r;i++)
        {
                for(j=0;j<c;j++)
                        printf("%d\t",a[i][j]);
                printf("\n");
        }
        printf("\nSECOND MATRIX IS :\n");
        for(i=0;i<r;i++)
        {
                for(j=0;j<c;j++)
                        printf("%d\t",b[i][j]);
                printf("\n");
        }
```

Addition of Matrices

```
printf("\nOPERATIONS THAT WE PERFORM ON MATRIX ARE :\n");
printf("1.ADDITION OF MATRICES\n2.SUBTRACTION OF A MATRICES\n3.MULTIPLICATION OF MATRICES\n4.QUIT\n");
        while(r>0)
        {
                printf("ENTER YOUR CHOICE : ");
                scanf("%d",&ch);
                switch(ch)
                {
                case 1:
                printf("\nADDITION OF MATRICES IS :\n");

                for(i=0;i<r;i++)
                {
                        for(j=0;j<c;j++)
                                c1[i][j]=a[i][j]+b[i][j];
                }
                for(i=0;i<r;i++)
                {
                        for(j=0;j<c;j++)
                                printf("%d\t",c1[i][j]);
                        printf("\n");
                }  break;
```

Matrix Subtraction

```
case 2:
                printf("\nSUBTRACTION OF MATRICES :\n");
                for(i=0;i<r;i++)
                {
                        for(j=0;j<c;j++)
                                c1[i][j]=a [i][j]-b[i][j];
                }
                for(i=0;i<r;i++)
                {
                        for(j=0;j<c;j++)
                                printf("%d\t",c1[i][j]);
                        printf("\n");
                }
                break;
```

Matrix Multiplication

```
case 3:
                printf("\nMATRIX MULTIPLICATION IS : \n");
                for(i=0;i<r;i++)
                {
                        for(j=0;j<c;j++)
                        {
                                c1[i][j]=0;
                                for(k=0;k<c;k++)
                                        c1[i][j]=c1[i][j]+a[i][k]*b[k][j];
                        }
                }
                for(i=0;i<r;i++)
                {
                        for(j=0;j<c;j++)
                                printf("%d\t",c1[i][j]);
                        printf("\n");
                }
                break;
                case 4:
                exit(0);
                break;
                default :
                        printf("\nINVALID CHOICE !! TRY AGAIN"); }
```